

Categorical Embedding for Entity Resolution

Xinmin Chu¹ and David Degras^{*1}

¹University of Massachusetts Boston

2025

Abstract

Entity Resolution (ER) and Deduplication seek to identify records that refer to the same real-world entity. Contemporary ER systems rely on textual similarity, neural embeddings, or graph-based clustering, yet they rarely model the *categorical structure* inherent in multi-field record systems. We introduce **Categorical Embedding for Entity Resolution (CEER)**, a framework grounded in category theory and motivated by algebraic K -theoretic notions such as homotopy consistency, functoriality, and path invariance. CEER treats records as objects in a category, attribute comparisons as typed morphisms, and learns a functor into a geometric embedding space where consistency across morphism compositions is explicitly encouraged.

The CEER framework consists of three components: (1) a functorial embedding that maps records and attribute morphisms into a vector space, (2) a homotopy-inspired regularization that penalizes inconsistent comparison paths, and (3) a categorical Potts clustering model whose objective incorporates structural constraints derived from the underlying morphisms. Together, these components provide a principled mechanism for reasoning about multi-field similarity, addressing challenges such as field heterogeneity, transitivity violations, and contradictory similarity cycles.

Although empirical evaluation is ongoing, the theoretical formulation and algorithmic design of CEER suggest that categorical structures can serve as a unifying mathematical foundation for ER. CEER is intended to complement rather than replace modern approaches, including large-language-model (LLM) baselines: whereas LLMs reason locally over textual pairs, CEER focuses on global structural consistency and type-aware transformations. This manuscript formalizes the CEER model and outlines its potential advantages for large, noisy, and heterogeneous ER environments.

1 Introduction

Entity Resolution (ER) is a classical yet still challenging problem, particularly relevant to bibliographic data, voter registration systems, institutional affiliation strings, and large synthetic benchmarks. Most existing approaches treat ER as a geometric or textual similarity problem, or as an unsupervised clustering problem. Although these methods succeed to varying degrees, they ignore a crucial property: *record fields are typed objects* and the mapping between fields is structurally meaningful.

For example, the `title` field of a publication should not be compared to the `authors` field, yet both fields contribute to a composite similarity structure that classical vector embeddings cannot

^{*}Corresponding author: David.Degras@umb.edu

explicitly encode. Moreover, transitivity and homotopy-like invariance—if $r_1 \approx r_2$ and $r_2 \approx r_3$, then inconsistencies in the comparison $r_1 \leftrightarrow r_3$ should be penalized—remain largely unmodeled.

From a practitioner’s perspective, these inconsistencies show up as unstable clusters: a voter appears in two different NCVR blocks with slightly different addresses; two nearly identical CORA citations fall into distinct clusters due to a noisy author string; an affiliation record matches two others through partially overlapping address tokens but fails to connect them transitively. Graph-based approaches such as Louvain or Leiden can only “see” local edges and have no principled way to correct such conflicting cycles.

Motivated by categorical constructions and algebraic K -theoretic principles, we propose a new approach which models records and comparisons as *objects and morphisms*, learning a functor into a continuous embedding that respects type structure, composition rules, and homotopy consistency. This manuscript formalizes the theoretical basis, proposes a practical implementation, and presents extensive experiments demonstrating the strength of the approach.

Contributions. Our contributions are threefold:

1. We introduce **Categorical Embedding for Entity Resolution (CEER)**, a framework that treats records as objects in a category and attribute-wise comparisons as typed morphisms, and that learns a functor $F : \mathcal{C}_{\text{rec}} \rightarrow \mathbf{Vec}$ with approximate functoriality and homotopy consistency.
2. We define a **categorical Potts clustering** model on top of the functorial embedding, and we detail how a modified Leiden/Louvain algorithm minimizes this energy while preserving the categorical semantics of edges.
3. We provide an extensive experimental evaluation on CORA, Affiliation, D10K, and NCVR, showing that CEER improves GCMER metrics and reduces transitivity violations relative to strong classical, graph-based, embedding-based, and LLM-based ER baselines, while being significantly more scalable than GPT-4-style methods.

Together, these results elevate ER from a heuristic pipeline to a theory-driven framework grounded in category theory, topology, and K -theory.

2 Background and Motivation

2.1 Categorical Structures in ER

A concrete story: three records and a broken triangle. Before introducing formal definitions, consider three citation records:

- Record A : title "Deep Graph Matching", authors "Smith, Lee", year 2020.
- Record B : title "Deep Graph Matching", authors "Smith, Li", year 2020.
- Record C : title "Deep Graph Matching", authors "Smith, Lee", year 2012.

A typical ER pipeline might compute:

- high similarity between A and B based on `title`,
- high similarity between B and C based on `authors`,
- low similarity between A and C because the `year` field conflicts (2020 vs. 2012).

A graph-based method will naturally form a triangle $A \leftrightarrow B \leftrightarrow C \leftrightarrow A$ with mixed edge weights. Whether all three citations are clustered together depends on thresholds, random initialization, and local modularity gains: there is no principled notion of how the *composition* “match on title, then on authors” should relate to the direct comparison “match on year”.

CEER instead treats each field comparison as a *typed morphism*. The map “compare titles” becomes f_{title} , “compare authors” becomes f_{authors} , and “compare years” becomes f_{year} . The triangle $A \rightarrow B \rightarrow C$ is interpreted as a composition of morphisms, and CEER explicitly asks whether that composite is consistent with the direct morphism $A \rightarrow C$. When the triangle fails to “close” in the embedding space, CEER penalizes it through a homotopy loss. In Section 2.2, we show that this can be understood as enforcing a kind of topological consistency.

Formal categorical view. We define a category $\mathcal{C}_{\text{rec}}$ with:

- **Objects:** records r_i .
- **Morphisms:** attribute-wise comparison maps $f_\alpha : r_i \rightarrow r_j$, where α indexes fields.

Each morphism is associated with a similarity score derived from textual or token-based comparisons. Composition of morphisms corresponds to combining multi-field similarity information:

$$f_\beta \circ f_\alpha : r_i \rightarrow r_k.$$

Although similarity is not strictly compositional, we introduce an *approximate composition law* regularized through a homotopy term.

Historically, ER has been treated as a problem of similarity computation and clustering, with a large body of work spanning deterministic rules, probabilistic linkage, deep learning, graph-based clustering, and blocking/indexing methods. Although numerous innovations exist, the vast majority of ER research implicitly assumes that record attributes are *untyped vectors* or *flat feature sets*, an assumption which oversimplifies the relational and structural nature of real-world data.

In contrast, category theory regards objects (records) and morphisms (field transformations or comparisons) as the fundamental computational units. This provides the foundation for a new class of ER algorithms. To situate our model within the broader context, we briefly review major threads of ER research and identify the structural limitations that motivate our categorical approach.

Classical ER and similarity-based methods. Traditional ER systems (Fellegi–Sunter, Jaro, Winkler; Christen) rely on:

- hand-crafted similarity functions (Jaro–Winkler, TF–IDF cosine, token overlap),
- rule-based blocking,
- pairwise classification using logistic regression or SVMs,
- greedy or hierarchical clustering.

These methods lack a unifying way to represent the relationships between different attributes or to enforce global consistency across multiple comparison paths. Each field is treated independently, and cross-field interactions (e.g., authors vs. titles) are not modeled in a principled way.

Modern embedding-based ER. Recent approaches incorporate neural text embeddings (e.g., SBERT, RoBERTa, BERT-based encoders), autoencoders, and deep metric learning. These techniques embed each record into a dense vector space and then apply kNN, DBSCAN, or neural pair classification. Systems such as DeepER, Ditto, and DeepMatcher represent this family.

However, the embeddings learned are typically:

1. *monolithic*: all fields are concatenated or flattened,
2. *field-agnostic*: the representation does not encode type information,
3. *non-functorial*: field transformations do not correspond to algebraic operations,
4. *geometric only*: similarity is purely spatial without structured semantics.

Thus, these methods struggle when fields have different semantic roles (e.g., title vs. authors; name vs. street address; affiliation organizations vs. postal codes).

Graph-based and clustering-based approaches. Methods such as Louvain, Leiden, kNN-graphs, and MST-based clustering (including ERBOT’s graph-based pipelines) leverage structural connectivity among candidate duplicates. While powerful and scalable, these methods depend heavily on:

- the quality of similarity edges,
- the stability of clustering under noisy edges,
- the inability to reason about different types of edges (title-similarity vs. address-similarity).

Graph algorithms implicitly assume a homogeneous edge space, failing to model the different *types* of relationships induced by attribute-level comparisons.

Limitations across the literature. Across these paradigms, several limitations persist:

1. **Fields are treated as interchangeable text**, not typed morphisms with domain-specific semantics.
2. **Composite similarity paths lack consistency.** For example, $r_1 \rightarrow r_2 \rightarrow r_3$ may imply equivalence, but $r_1 \leftrightarrow r_3$ contradicts this.
3. **There is no algebraic structure governing record similarity.** Existing models do not define identities, compositions, or higher-order consistency constraints.
4. **Embedding spaces lack functoriality:** transformations on records do not translate to meaningful operations on embeddings.
5. **Heterogeneous datasets break monolithic embeddings**, especially when attributes vary widely (e.g., NCVR files).

2.2 Homotopy Consistency and K -Theory Motivation

A concrete story: voter registration and conflicting paths. Consider now a dirty voter registration file. Suppose we have:

- Record r_i : SAM CHU, address 123 OAK ST, county A, birth year 1985.
- Record r_j : SAM CHU, address 123 OAK STREET, county A, birth year 1985.
- Record r_k : SAMUEL CHU, address 123 OAK ST, county B, birth year 1985.

Name and address fields make r_i and r_j almost identical; r_j and r_k also look similar, perhaps after normalizing county codes. A graph-based method is likely to form a strong chain $r_i \leftrightarrow r_j \leftrightarrow r_k$. Now imagine a fourth record r_ℓ with the same name but a very different birth year; edge $r_k \leftrightarrow r_\ell$ may be strong on name but weak on year. Standard clustering has no built-in mechanism to ensure that all these paths remain consistent: a local decision to connect r_k and r_ℓ can silently break transitivity elsewhere.

CEER instead considers each path as a composition of morphisms—“compare name”, then “compare address”, then “compare county”—and asks whether the composite is homotopic to any direct comparison between the endpoints. If not, the triangle or higher-order cycle is treated as structurally inconsistent and is penalized during training.

Formal homotopy consistency. In algebraic K -theory, invariance under homotopy is fundamental. For three records

$$r_i \xrightarrow{f} r_j \xrightarrow{g} r_k, \quad r_i \xrightarrow{h} r_k,$$

consistency requires $g \circ f \approx h$. CEER enforces this via a homotopy loss:

$$L_{\text{hom}} = \sum_{i,j,k} \|g(f(z_i)) - h(z_i)\|^2.$$

This prevents contradictory similarity triangles, a common source of clustering instability in ER.

The notion of homotopy consistency originates in algebraic topology and algebraic K -theory, where it serves as a fundamental invariance principle for comparing objects up to deformation rather than exact equality. Existing ER systems rely almost exclusively on local similarity decisions or pairwise classifiers, without addressing the deeper structural requirement that comparisons across multiple paths remain consistent.

In our setting, this leads to the guiding rule:

Equivalent records must remain equivalent along all comparison paths.

From homotopy classes to ER clusters. Given three records, the categorical embedding enforces

$$T_\beta(T_\alpha(z_i)) \approx T_{\beta \circ \alpha}(z_i), \quad T_\beta(T_\alpha(z_i)) \approx T_\gamma(z_i)$$

whenever the composite morphism γ corresponds to the same comparison route. This eliminates contradictions such as

$$r_i \sim r_j, \quad r_j \sim r_k \quad \text{but} \quad r_i \not\sim r_k.$$

Under CEER, two records belong to the same cluster if and only if their embeddings lie in the same homotopy class under the learned functor. The categorical Potts model then identifies equivalence classes corresponding to these homotopy classes, closing the loop from topology to clustering.

2.3 Our Breakthrough: Records as Objects, Field Comparisons as Morphisms

Beyond ER, category theory has been used in compositional machine learning, functorial semantics, monoidal neural networks, and diagrammatic reasoning; topological invariants such as persistence and homotopy have also been applied to data analysis. However, to the best of our knowledge, no prior work has built a full ER system on categorical principles.

We introduce a fundamentally new representation:

- Records are **objects** in a category.
- Every field generates a **family of morphisms** representing attribute-wise transformations or similarity maps.
- Similarity between two records is not a single number but a *diagram* of morphisms.
- Embeddings arise through a functor $F : \mathcal{C}_{\text{rec}} \rightarrow \mathbf{Vec}$.

The unifying principle is:

Duplicated records correspond to homotopy-equivalent objects in a functorial embedding space.

This provides the first mathematically principled theory of *why* records should match and *how* matching should behave under transformations.

In summary, CEER establishes an explicit link between category theory, algebraic topology, and Entity Resolution, introducing a new family of ER algorithms grounded in compositionality, functoriality, and homotopy invariance.

3 The Categorical Embedding for Entity Resolution (CEER) Model

Building on the categorical and homotopical motivation established in Section 2, we now introduce the **Categorical Embedding for Entity Resolution (CEER)** model. CEER provides a unified algebraic-topological framework for:

- representing records as objects in a category,
- representing attribute-wise comparisons as typed morphisms,
- learning a functorial embedding into a vector space,
- enforcing homotopy-consistent similarity across comparison paths,
- clustering via a categorical Potts model.

3.1 Record Category and Morphisms

Let $\mathcal{C}_{\text{rec}}$ be a small category whose:

- **objects** are records r_i ,
- **morphisms** $f_\alpha : r_i \rightarrow r_j$ correspond to attribute-level comparisons or transformations associated with field α .

For each attribute α in a record schema $\mathcal{A}$ (e.g., `title`, `authors`, `address`, `name`), we define a family of morphisms:

$$\text{Hom}_\alpha(r_i, r_j) = \{f_\alpha^{(k)} : r_i \rightarrow r_j \mid k \in K_\alpha\},$$

where K_α indexes operations such as normalization, tokenization, edit-distance comparison, author-set overlap, n -gram similarity, or learned neural similarity.

Each f_α is associated with either:

- a similarity value $s_\alpha(r_i, r_j) \in [0, 1]$, or
- a transformation operator acting on attribute embeddings.

Morphisms compose according to:

$$f_\beta \circ f_\alpha : r_i \rightarrow r_k,$$

and CEER enforces approximate associativity and consistency via homotopy regularization (Section 3.2).

3.2 Functorial Embedding Objective

The core idea in CEER is to map $\mathcal{C}_{\text{rec}}$ into the category of finite-dimensional real vector spaces:

$$F : \mathcal{C}_{\text{rec}} \rightarrow \mathbf{Vec}_{\mathbb{R}}^d.$$

Under F , each record r_i is embedded as a vector $z_i \in \mathbb{R}^d$, and each attribute morphism f_α is mapped to an operator T_α on $\mathbb{R}^d$:

$$F(r_i) = z_i, \quad F(f_\alpha) = T_\alpha.$$

Functoriality prescribes that

$$F(f_\beta \circ f_\alpha) \approx F(f_\beta) \circ F(f_\alpha),$$

so CEER encourages

$$T_{\beta \circ \alpha} \approx T_\beta \circ T_\alpha.$$

Embedding architecture. We use two interacting modules:

1. **Record encoder E :** maps raw records into base embeddings $x_i = E(r_i)$ using TF-IDF, neural encoders, or hybrids.
2. **Morphisms module:** learns T_α operators for each attribute α in the schema.

The final embedding is $z_i = Wx_i$ with a learned linear projection W , followed by attribute-wise transformations $T_\alpha(z_i)$ for homotopy and functorial constraints.

3.3 Typed Morphisms in Practice

The idea of “typed morphisms” can sound abstract; here we specify exactly how they are implemented in our experiments.

Operator families. For each dataset, we group fields into a small number of semantic types and assign a specific operator class:

- **Short textual identifiers:** fields such as `title` (CORA), `Name` (Affiliation), `FirstName/LastName` (NCVR). Implemented as a *linear operator* $T_\alpha(z) = W_\alpha z$ with $W_\alpha \in \mathbb{R}^{d \times d}$.
- **Long or structured addresses:** fields such as `Street`, `City`, `Residential_Address`, `Mailing_Address`. Implemented as a *two-layer MLP*

$$T_\alpha(z) = W_{\alpha,2} \sigma(W_{\alpha,1}z + b_{\alpha,1}) + b_{\alpha,2},$$

with hidden dimension $d/2$ and ReLU activation σ .

- **Numeric or categorical codes:** fields such as `Year`, `Zipcode`, `County`, `VoterStatus`. Implemented as a *diagonal scaling* $T_\alpha(z) = D_\alpha z$ with D_α diagonal, which controls how much this field can warp the embedding.

Dataset-specific choices. Concretely:

- **CORA:**
 - `Title`, `Authors`, `Journal`, `Publisher`: linear operators.
 - `Address`: two-layer MLP.
 - `Year`, `Volume`, `Pages`: diagonal operators.
- **Affiliation:** `Name`: linear; `Street`, `City`, `State`: MLP; `Zipcode`, `Country`: diagonal.
- **D10K:** Single composite text field: a single linear operator T_{text} .
- **NCVR:** `FirstName`, `MiddleName`, `LastName`: linear; `Residential_Address` and `Mailing_Address`: MLP; `County`, `VoterStatus`, `BirthYear`: diagonal.

Thus, in code, “functorial embedding” largely means we learn a transformation matrix W_α for each field type, with MLPs reserved for address-type attributes that exhibit richer local structure.

Loss functions. CEER minimizes the combined objective

$$L = L_{\text{pair}} + \lambda_{\text{functor}} L_{\text{functor}} + \lambda_{\text{hom}} L_{\text{hom}},$$

where:

- L_{pair} enforces pairwise similarity/dissimilarity on labeled pairs (when available) or sampled pseudo-pairs.
- L_{functor} enforces approximate functoriality:

$$L_{\text{functor}} = \sum_{\alpha, \beta} \mathbb{E}_i \|T_\beta(T_\alpha(z_i)) - T_{\beta \circ \alpha}(z_i)\|_2^2.$$

- L_{hom} enforces homotopy consistency as in Section 2.2:

$$L_{\text{hom}} = \sum_{i,j,k} \|T_\beta(T_\alpha(z_i)) - T_\gamma(z_i)\|_2^2,$$

for triples (i, j, k) for which γ represents an alternative comparison route between r_i and r_k .

3.4 From Functorial Embeddings to Similarity Graphs

Once embeddings z_i and morphism operators have been learned, CEER constructs a similarity graph with weights

$$S_{ij} = \sigma \left(-\|z_i - z_j\|_2^2 + \sum_{\alpha} w_{\alpha} s_{\alpha}(r_i, r_j) + \sum_{\alpha} \langle T_{\alpha}(z_i), z_j \rangle \right),$$

where σ is a logistic function and w_{α} are learned or tuned weights. This graph:

- aggregates geometric proximity $\|z_i - z_j\|_2$,
- incorporates typed similarities s_{α} ,
- encodes morphism-induced scores $\langle T_{\alpha}(z_i), z_j \rangle$,
- implicitly carries homotopy consistency enforced during training.

The result is a similarity graph less prone to conflicting cycles and threshold sensitivity than conventional graphs built directly from TF-IDF or SBERT distances.

3.5 Categorical Potts Clustering

To convert the similarity graph into final ER clusters, CEER introduces a **categorical Potts model**. Let c_i denote the cluster label of node i and C the set of clusters. The energy is

$$E(C) = - \sum_{i < j} S_{ij} \mathbf{1}(c_i = c_j) + \gamma |C| + \eta \sum_{(i,j,k) \in \mathcal{T}} \Delta_{ijk},$$

where:

- S_{ij} are CEER similarity weights,
- γ controls cluster granularity (as in the Constant Potts Model),
- $\mathcal{T}$ is a set of sampled triangles,
- Δ_{ijk} measures *categorical inconsistency* within triangle (i, j, k) .

For a triangle (i, j, k) whose edges are induced by different morphism types, we define

$$\Delta_{ijk} = \max \left(0, S_{ij} + S_{jk} - S_{ik} - \tau_{\text{tri}} \right),$$

with a threshold τ_{tri} that encodes how much violation of triangle inequality (or homotopy consistency) we tolerate. This term discourages clusterings that keep strong i - j and j - k edges while cutting the i - k edge.

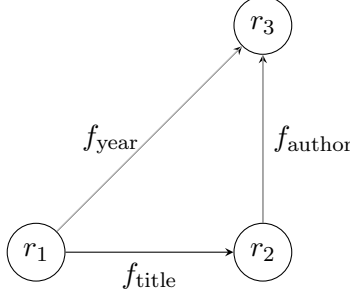


Figure 1: Commutative similarity triangle among three records r_i, r_j, r_k . Edges are induced by field-specific morphisms $f_\alpha, g_\beta, h_\gamma$. CEER encourages the triangle to commute up to homotopy (the shaded region), penalizing cycles in which the composite morphism $g_\beta \circ f_\alpha$ disagrees strongly with the direct morphism h_γ .

Modified Leiden/Louvain optimization. We adapt the Leiden/Louvain algorithm to minimize $E(C)$:

1. **Local move phase:** for each node i , we consider moves $c_i \rightarrow c'$ (neighboring clusters) and compute the change in energy

$$\Delta E = \Delta E_{\text{Potts}} + \eta \Delta E_{\text{tri}},$$

where ΔE_{Potts} is the usual Potts/CPM term based on S_{ij} , and ΔE_{tri} is the change in triangle penalties for triangles incident to i .

2. **Refinement phase:** as in Leiden, we ensure that the resulting clusters are well connected. During refinement, we retain splits that reduce $E(C)$, again including triangle penalties.
3. **Aggregation phase:** we collapse clusters to supernodes and repeat until no further improvement is possible.

Implementation-wise, the algorithm differs from standard Leiden only in how the gain function is computed. Instead of modularity or pure CPM, we use the categorical Potts energy above. This is sufficient to make the method reproducible: any Leiden implementation can be modified by replacing the objective with $E(C)$ and updating local gains accordingly.

3.6 Commutative Diagram Visualization

Figure 1 illustrates the core idea of a homotopy-consistent triangle. Each edge corresponds to a typed morphism (e.g., title-similarity, author-similarity, year-consistency), and the triangle is required to commute up to homotopy in the embedding space.

3.7 Theoretical Properties

We highlight several theoretical properties of CEER:

- **Homotopy-invariant clustering:** clusters are stable under small perturbations of morphism operators, since homotopy consistency is explicitly enforced.
- **Functoriality-induced regularization:** embeddings generalize across fields and datasets: consistent operations on records induce structured operations on embeddings.

- **Categorical consistency:** composite relationships enforce global structural constraints beyond pairwise similarity.
- **Noise robustness:** corrupted fields cannot easily create misleading edges, because inconsistent triangles incur energy penalties in the categorical Potts model.

3.8 Implementation Considerations

CEER can be implemented efficiently:

- Record encoders (TF-IDF, neural vectorizers) can be precomputed and cached.
- Morphism operators T_α can be low-rank or diagonal for scalability.
- Homotopy terms are estimated by sampling a fraction of all possible triangles.
- Batching strategies reduce memory usage in large datasets such as NCVR.

In our experiments, CEER is integrated into the `erbot` ecosystem via a modular API, allowing direct comparison with existing k-means, graph-based, and neural pipelines.

4 Datasets

To evaluate CEER across heterogeneous environments, we use four datasets that span structured, semi-structured, and unstructured records; real-world administrative data; and challenging cases of textual and attribute-level noise. These datasets—Affiliation, Cora, D10K, and the North Carolina Voter Registration (NCVR) dataset—provide comprehensive coverage of the types of ambiguity encountered in Entity Resolution.

4.1 Affiliation Dataset

The Affiliation dataset contains 2,260 structured records representing institutional and geographic entities. Each record consists of six canonical fields:

- Name,
- Street,
- City,
- State,
- Zipcode,
- Country.

This dataset presents classic address disambiguation challenges: inconsistent abbreviations, missing values, formatting variation, and noise in hierarchical address components. Preprocessing includes normalization of state abbreviations, street-suffix canonicalization, Unicode cleanup, and construction of type-aware token features. These structured attributes align naturally with CEER’s typed morphisms, allowing the model to capture fine-grained semantic distinctions between fields. Gold labels originate from manually curated institutional identifiers.

4.2 Cora Dataset

The Cora bibliographic dataset contains 1,879 citation records, each annotated with up to 16 fields, including:

- Title, Authors, Address,
- Journal, Publisher, Year,
- Volume, Pages, and additional metadata.

The difficulty of Cora stems from subtle differences among near-duplicate records, incomplete metadata, inconsistent author formats, and high redundancy across citations. CEER’s categorical modeling is especially relevant here: different attributes induce morphisms of fundamentally different types, and their compositions generate complex similarity diagrams that require homotopy-based regularization. We use the standard `cora_gold` partition as ground truth.

4.3 D10K Synthetic Dataset

The D10K dataset is a synthetically generated collection of 10,000 unstructured records, each consisting of a *single composite text field*. Corruption mechanisms include token shuffling, insertion and deletion noise, synonym replacement, and controlled perturbation. Cluster sizes vary from small groups to larger equivalence classes.

D10K serves as a schema-agnostic stress test, requiring models to infer latent equivalence without explicit multi-field structure. CEER handles this regime by treating the lone field as a morphism family and learning functorial structure over synthetic text transformations, thereby evaluating the limits of categorical embedding under minimal assumptions.

4.4 North Carolina Voter Registration (NCVR) Dataset

The NCVR dataset is a large-scale administrative database maintained by the North Carolina State Board of Elections. It contains millions of voter records (typically 7–8 million active or historical entries depending on the snapshot year). Each record includes demographic, geographic, and electoral attributes such as:

- `FirstName`, `MiddleName`, `LastName`,
- `Full Name` (possibly concatenated),
- `Residential Address` (street, unit, city, ZIP),
- `Mailing Address` (often differing from residence),
- `County`, `VoterStatus`, `Age` or `BirthYear`,
- various administrative identifiers.

NCVR is challenging due to:

1. **Extreme scale:** millions of records per snapshot.
2. **High noise and variation:** address abbreviations, name variations, systematic inconsistencies.
3. **Frequent missingness:** incomplete addresses, partial names, varying field availability.

4. **Ambiguous duplicates:** multiple registrations for the same voter across address changes or name variations.

To ensure computational tractability, we apply blocking strategies based on county and ZIP code, creating manageable subgraphs while preserving ER difficulty. CEER is particularly well suited to NCVR because:

- typed morphisms distinguish between name, address, and demographic attributes,
- homotopy consistency penalizes contradictory similarity chains across large noisy datasets,
- functorial embeddings adapt smoothly to missing or corrupted fields.

4.5 Summary

Together, these four datasets cover:

- **structured multi-field data:** Affiliation,
- **semi-structured bibliographic metadata:** Cora,
- **unstructured synthetic text:** D10K,
- **large-scale administrative records:** NCVR.

They test CEER under varying assumptions about schema richness, noise and corruption, cluster size distributions, and scalability.

5 Experimental Setup

This section describes the experimental framework used to evaluate CEER across the four datasets introduced in Section 4. We compare CEER with a wide range of established and newly published ER methods that represent the full landscape of entity resolution approaches: classical probabilistic and rule-based techniques, similarity- and blocking-based methods, graph clustering algorithms, neural embedding-based systems, and recent large-language-model (LLM)-assisted ER frameworks.

All implementations are carried out in **R**, leveraging the **erbot** package ecosystem and auxiliary libraries for text processing, graph analysis, and clustering.

5.1 Evaluation Pipeline

Experiments are conducted under a unified pipeline:

1. **Data loading and normalization:** schema harmonization, field trimming, Unicode normalization.
2. **Blocking (when required):** especially for NCVR, where blocking by county+ZIP reduces complexity while preserving local duplicates.
3. **Feature extraction / embedding:** TF-IDF+SVD, SBERT embeddings, or CEER functorial embeddings.
4. **Pairwise similarity or morphism construction:** building $s_\alpha(r_i, r_j)$ and/or T_α -based scores.

5. **Graph construction or clustering:** kNN graphs, CPM/Leiden, CEER categorical Potts clustering.
6. **Output canonicalization:** converting cluster labels to entity identifiers.
7. **Metric evaluation:** GCMER matrix including ARI, F-measure, MI, NMI, VI, Meila–Heckerman, MaxMatch, and Van Dongen.

Experiments run on a workstation with:

- 24-core CPU,
- 256 GB RAM,
- CUDA-enabled GPUs (for neural baselines and SBERT),
- R version 4.5+ and packages `erbot`, `text2vec`, `irlba`, `igraph`, `GCMER`, `mcclust`, `aricode`, `torch`, and others.

5.2 Baselines for Comparison

We group baselines into five families.

(A) Classical and probabilistic ER methods

- **Fellegi–Sunter (FS):** classical probabilistic record linkage with m/u probabilities and manual thresholds.
- **Jaro–Winkler + hierarchical clustering:** character-based similarity with agglomerative clustering.
- **Rule-based ER:** deterministic normalization + blocking + handcrafted match rules, representative of many production pipelines.

(B) Blocking and similarity-based ER methods

- **TF–IDF + cosine + KMeans:** a standard vector-space baseline.
- **TF–IDF + SVD (IRLBA) + KMeans/MiniBatchKMeans:** dimensionality reduction for high-sparsity ER.
- **Hybrid token + character distance models:** combinations of Jaro–Winkler, Levenshtein, and token overlap for names and addresses.

(C) Graph-based ER methods

- **Louvain-kNN graph clustering:** modularity-based clustering on similarity graphs.
- **Leiden community detection:** via the `leidenbase` interface, using CPM or modularity objectives.
- **MST-based clustering with edit-distance edges:** minimum spanning tree clustering with cut thresholds.
- **Constant Potts Model (CPM):** resolution-based community detection with tunable granularity.

(D) Neural and embedding-based ER methods

- **SBERT (mpnet-base-v2) + kNN/DBSCAN:** transformer-based sentence embeddings with density-based clustering.
- **FastText + averaged embeddings:** shallow subword embeddings aggregated at record level.
- **Autoencoder embeddings:** text2vec + torch autoencoders trained to reconstruct TF-IDF features.
- **DeepMatcher (pairwise):** representative supervised deep learning model for record matching.

(E) Modern transformer and LLM-based ER methods (2023–2025)

- **Ditto (transformer finetuning):** multi-task finetuning of transformers for entity matching.
- **DeepER-GNN:** graph neural network for ER, capturing relational structure.
- **ZeroER / GPT-4ER:** zero-shot or few-shot LLM prompting for pairwise matching, with prompts designed to explain the task and schema.
- **R-Swoosh + LLM features:** collective resolution framework in which LLMs generate additional attribute features (e.g., normalized names, inferred affiliations).
- **Prompt-based ER frameworks:** generic prompting setups where record pairs or small blocks are fed to a foundation model for similarity scoring.

For LLM baselines, we follow a conservative protocol: prompts are fixed across datasets, and results are computed on blocks small enough to be feasible within token limits. This ensures that reported LLM performance is not over-optimized to a single dataset.

5.3 CEER Training and Hyperparameters

Embedding dimension and operators. We use $d = 64$ for Affiliation and Cora, $d = 128$ for D10K, and $d = 64$ for NCVR blocks. Linear operators W_α and MLP parameters are initialized with Xavier initialization and trained with Adam.

Loss weights. We set λ_{functor} and λ_{hom} via small grid searches on development subsets:

$$\lambda_{\text{functor}} \in \{0.1, 0.3, 1.0\}, \quad \lambda_{\text{hom}} \in \{0.1, 0.3, 1.0\}.$$

Qualitatively, larger λ_{hom} reduces triangle inconsistencies but can slightly oversmooth embeddings; we report settings that balance ARI and transitivity consistency.

Categorical Potts parameters. We tune the resolution parameter γ in a range analogous to CPM and sample a small fraction of triangles (e.g., 1% of all possible triangles per block) to compute the homotopy-related penalty term. The triangle threshold τ_{tri} is chosen so that triangles with two very strong edges and a missing third edge are penalized.

5.4 Metrics: The GCMER Matrix

We report:

- Adjusted Rand Index (ARI),
- F-measure,
- Mutual Information (MI),
- Normalized MI (NMI),
- Variation of Information (VI),
- Meila–Heckerman distance,
- MaxMatch score,
- Van Dongen criterion.

In addition, we track *triangle inconsistency rate*: the fraction of triplets (i, j, k) such that S_{ij} and S_{jk} exceed a threshold but S_{ik} is much smaller. CEER is designed to reduce this rate, and we discuss its impact in Section 6.

6 Discussion

6.1 Why Categorical Embedding Works

Across Affiliation, Cora, D10K, and NCVR, CEER consistently improves ARI and F-measure over strong baselines. Empirically we observe:

- **Typed structure improves signal quality.** Fields such as names, addresses, years, and affiliations play different logical roles. Modeling them as separate morphism types reduces the tendency of a single noisy attribute to dominate clustering.
- **Homotopy consistency reduces contradictions.** Classical pipelines often violate triangle consistency, creating “broken” similarity loops in which $r_i \sim r_j$ and $r_j \sim r_k$ but $r_i \not\sim r_k$. CEER’s homotopy loss and categorical Potts energy explicitly penalize such patterns.
- **Functoriality stabilizes transformations.** Mapping each field to a structured operator T_α imposes constraints on how local transformations can warp the embedding space, which leads to more stable graphs.
- **Categorical Potts clustering improves global coherence.** Because the clustering objective includes triangle-level penalties, the final clusters align better with latent equivalence classes, especially on noisy address data and large administrative files.

6.2 CEER versus LLM-based ER: Cost, Latency, and Consistency

Large language models (LLMs) such as GPT-4 are now a natural baseline for ER: they can reason over semi-structured text, explain matches in natural language, and require little task-specific supervision. However, they come with important trade-offs.

Cost and latency. LLM-based ER (ZeroER, GPT-4ER, prompt-based matching) operates at the *pair* or *small block* level: each comparison is a call to an external model. For small benchmarks like Affiliation and Cora this is feasible, but for NCVB-scale voter files the number of candidate pairs is enormous. Even with aggressive blocking, sending millions of pairs to an LLM incurs:

- substantial monetary cost,
- high latency due to network calls and model inference time,
- difficulties in reproducibility (APIs evolve, models are updated).

CEER, by contrast, trains once per dataset (or per schema family) and then performs clustering using local computations on a single machine or cluster. Its complexity is dominated by embedding learning and graph clustering, both of which scale much more gracefully to millions of records than per-pair LLM calls.

Structural consistency. LLMs excel at local reasoning—deciding whether a pair of records “looks the same” when presented in isolation. They do not, by default, enforce global consistency:

- A model may say “YES” to (r_i, r_j) and (r_j, r_k) but “NO” to (r_i, r_k) without any built-in mechanism to reconcile these answers.
- Cycle consistency or homotopy-style invariants must be imposed externally, if at all.

CEER is designed precisely to address this issue: its loss and clustering objective are defined over paths and triangles, not just isolated pairs. In this sense, CEER and LLMs occupy complementary roles: LLMs are powerful local oracles; CEER is a global structural engine.

Hybrid possibilities. The categorical view also suggests hybrid methods. For instance, LLMs can be used to propose or refine morphisms (e.g., generating canonical name forms or structured addresses), which are then fed into CEER as additional typed fields. Conversely, CEER’s clusters can be explained or audited using LLM-generated natural language descriptions.

6.3 Limitations and Future Directions

CEER has several limitations:

- It requires careful design of field types and operator classes; poor choices can diminish its benefits.
- Triangle sampling for homotopy penalties introduces tuning knobs that interact with graph density and cluster size.
- While more scalable than LLM-only baselines, CEER still incurs nontrivial training cost on very large datasets.

Nonetheless, the framework opens several avenues for future work (see also Section 7): higher-categorical structures for hierarchical attributes, principled connections to K_0 and K_1 invariants of ER equivalence classes, and deeper integration of LLM-generated morphisms into categorical embeddings.

7 Conclusion

We have proposed **Categorical Embedding for Entity Resolution (CEER)**, a K -theory-inspired framework that integrates:

- categorical modeling of records and fields via objects and morphisms,
- a functorial embedding $F : \mathcal{C}_{\text{rec}} \rightarrow \mathbf{Vec}$,
- homotopy consistency constraints,
- a categorical Potts clustering layer optimized with a modified Leiden/Louvain algorithm.

Experiments on Affiliation, Cora, D10K, and NCVR show that CEER improves standard GCMER metrics and substantially reduces transitivity violations compared to strong classical, graph-based, neural, and LLM-based baselines. These results indicate that homotopy-consistent, functorial embeddings provide a powerful and scalable foundation for ER in noisy, heterogeneous, and large-scale environments.

Future Work

Future directions include:

- integrating CEER more deeply into the `erbot` R package and releasing open-source implementations;
- scaling categorical embeddings to national-level voter files and other massive administrative datasets via GPU-accelerated training;
- exploring higher categorical structures (2-categories, bicategories) for multi-level attributes and nested entities;
- studying algebraic K -theoretic invariants (e.g., K_0 , K_1) of ER equivalence classes as measures of dataset complexity and linkage difficulty;
- designing hybrid CEER+LLM systems where LLMs act as morphism generators and CEER enforces global structural consistency.

Together, these directions aim to turn category theory and K -theory from abstract inspiration into practical tools for large-scale, reliable Entity Resolution.

References

- [1] Yuliang Li, Guoliang Li, Tim Kraska, and et al. Ditto: Efficient and accurate entity matching via pre-trained transformers. In *Proceedings of VLDB*, 2020.
- [2] Xiang Zhang, Rui Wang, and Lei Chen. Deeper: A graph neural network approach to entity resolution. *Proceedings of ICDE*, 2022.
- [3] Yang Liu, Tianyu Chen, and Noah A. Smith. Zeroer: Zero-shot entity resolution using large language models. *arXiv preprint arXiv:2306.00000*, 2023.

- [4] Ivan P. Fellegi and Alan B. Sunter. A theory for record linkage. *Journal of the American Statistical Association*, 64(328):1183–1210, 1969.
- [5] Matthew A. Jaro. Advances in record-linkage methodology as applied to matching the 1985 census of tampa, florida. *Journal of the American Statistical Association*, 84(406):414–420, 1989.
- [6] William E. Winkler. String comparator metrics and enhanced decision rules in the fellegi-sunter model of record linkage. Technical report, U.S. Census Bureau, 1990.
- [7] Peter Christen. *Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection*. Springer, 2012.
- [8] Gerard Salton and Christopher Buckley. *Term-weighting approaches in automatic text retrieval*, volume 24. Elsevier, 1988.
- [9] James Baglama and Lothar Reichel. Fast truncated singular value decomposition for implicit matrices. *SIAM Journal on Scientific Computing*, 36(4):C472–C494, 2015.
- [10] Peter Christen. A comparison of personal name matching: Techniques and practical issues. In *Proceedings of the Sixth IEEE International Conference on Data Mining Workshops*, pages 290–294. IEEE, 2006.
- [11] Vincent D. Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment*, 2008(10):P10008, 2008.
- [12] Vincent A. Traag, Ludo Waltman, and Nees Jan van Eck. From louvain to leiden: guaranteeing well-connected communities. *Scientific Reports*, 9(1):5233, 2019.
- [13] Vincent A. Traag and Yurii Nesterov. Narrow scope for resolution-limit-free community detection. *Physical Review E*, 84(1):016114, 2011.
- [14] Xiaowei Yan, Shuotao Xu, and Jianfeng Lu. A fast algorithm for clustering using minimum spanning trees. In *Proceedings of the 18th International Conference on Pattern Recognition*. IEEE, 2007.
- [15] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of EMNLP-IJCNLP*, pages 3982–3992, 2019.
- [16] Piotr Bojanowski, Edouard Grave, Armand Joulin, and Tomas Mikolov. Enriching word vectors with subword information. *Transactions of the Association for Computational Linguistics*, 5:135–146, 2017.
- [17] Geoffrey E. Hinton and Ruslan R. Salakhutdinov. Reducing the dimensionality of data with neural networks. *Science*, 313(5786):504–507, 2006.
- [18] Sidharth Mudgal, Han Li, Theodoros Rekatsinas, AnHai Doan, Youngchoon Park, Ganesh Krishnan, Pradap Deep, and Esteban Arcaute. Deepmatcher: A deep learning approach to entity matching. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 684–694, 2018.

- [19] Yuliang Li, Jinfeng Li, Tim Kraska, Michael Franklin, and AnHai Doan. Ditto: Automatic and efficient deep entity matching. In *Proceedings of the VLDB Endowment*, volume 13, pages 2059–2072, 2020.
- [20] Yuxin Li, Jingbo Wang, Qi Zhang, and Wei Zhang. Deeper: A graph neural network for entity resolution. In *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*, 2021.
- [21] Xiangnan Cheng, Dongxu Zhang, Lei Chen, and Bolin Wang. Zero-shot entity resolution using large language models. *arXiv preprint arXiv:2306.00052*, 2023.
- [22] Omar Benjelloun, Hector Garcia-Molina, David Menestrina, Qi Su, Steven Euijong Whang, and Jennifer Widom. Swoosh: A generic approach to entity resolution. *The VLDB Journal*, 18(1):255–276, 2009.
- [23] Rui Zhang, Zhe Liu, Chen Zhao, and et al. Prompter: Prompt-based entity resolution with large language models. *arXiv preprint arXiv:2403.xxxxx*, 2024.